

KONVERTO

passion for technology



Intelligent Spring Boot Apps

Text, Images & Audio
with Spring AI

About me



Peter Unterthurner

Konverto Lab – Full Stack Software Engineer

Email: peter.unterthurner@konverto.eu
Homepage: aboutme.thurnaround.de
Github: github.com/Jjemus

Agenda

- Spring AI
- Features
 - Text
 - Structured Output
 - Images
 - Audio
 - Data Enrichment
 - Tool Calling
- Reliability
 - Observability
 - Evaluation Testing
- Benefits & Caveats

Spring AI

Spring AI

What is it?

- Extension for Spring Ecosystem
- Seamless integration into Spring Boot
- Provides Spring-typical access to LLM's and generative AI

Spring AI

Model comparison

Provider	Multimodality	Tools/Functions	Streaming	Retry	Observability	Built-in JSON	Local	OpenAI API Compatible
Anthropic Claude	text, pdf, image	✓	✓	✓	✓	✗	✗	✗
Azure OpenAI	text, image	✓	✓	✓	✓	✓	✗	✓
DeepSeek (OpenAI-proxy)	text	✓	✓	✓	✓	✓	✓	✓
Google VertexAI Gemini	text, pdf, image, audio, video	✓	✓	✓	✓	✓	✗	✓
Groq (OpenAI-proxy)	text, image	✓	✓	✓	✓	✗	✗	✓
HuggingFace	text	✗	✗	✗	✗	✗	✗	✗
Mistral AI	text, image, audio	✓	✓	✓	✓	✓	✗	✓
MiniMax	text	✓	✓	✓	✓	✗	✗	✓
Moonshot AI	text	✓	✓	✓	✓	✓	✗	
NVIDIA (OpenAI-proxy)	text, image	✓	✓	✓	✓	✗	✗	✓
OCI GenAI/Cohere	text	✗	✗	✗	✓	✗	✗	✗
Ollama	text, image	✓	✓	✓	✓	✓	✓	✓
OpenAI	In: text, image, audio Out: text, audio	✓	✓	✓	✓	✓	✗	✓
Perplexity (OpenAI-proxy)	text	✗	✓	✓	✓	✗	✗	✓
QianFan	text	✗	✓	✓	✓	✗	✗	✗
ZhiPu AI	text, image, docs	✓	✓	✓	✓	✗	✗	✗
Amazon Bedrock Converse	text, image, video, docs (pdf, html, md, docx ...)	✓	✓	✓	✓	✗	✗	✗

Text

Text

Text Generation - Blocking

```
@RestController
public class RecipeController {
    private final ChatClient chatClient;

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    @GetMapping("recipe")
    String recipe() {
        return this.chatClient.prompt()
            .user("Give me a recipe for a delicious dish")
            .call()
            .content();
    }
}
```

Text

Text Generation - Stream

```
@RestController
public class RecipeController {
    private final ChatClient chatClient; 2 usages

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    @GetMapping("recipe/stream")
    public Flux<String> recipeStream() {
        return chatClient
            .prompt() ChatClientRequestSpec
            .user(text: "Give me a recipe for a delicious dish")
            .stream() StreamResponseSpec
            .content();
    }
}
```

Text

Text Generation – System Instructions

```
@RestController
public class RecipeController {
    private final ChatClient chatClient; 2 usages

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder
            .defaultSystem(text: "You are a helpful assistant that provides recipes.")
            .build();
    }

    @GetMapping("recipe")
    public String recipe() {
        return chatClient
            .prompt() ChatClientRequestSpec
            .user(text: "Give me a recipe for a delicious dish")
            .call() CallResponseSpec
            .content();
    }
}
```

Text

Text Generation - Memory

```
@RestController
public class RecipeController {
    private final ChatClient chatClient;
    private final PromptChatMemoryAdvisor promptChatMemoryAdvisor;

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();

        var memory =
            MessageWindowChatMemory.builder()
                .chatMemoryRepository(new InMemoryChatMemoryRepository())
                .build();
        this.promptChatMemoryAdvisor = PromptChatMemoryAdvisor.builder(memory).build();
    }

    @GetMapping("recipe/last")
    public String recipeLast() {
        return chatClient
            .prompt()
            .advisors(promptChatMemoryAdvisor)
            .user(text: "What was the last recipe I asked for?")
            .call()
            .content();
    }
}
```

Structured Output

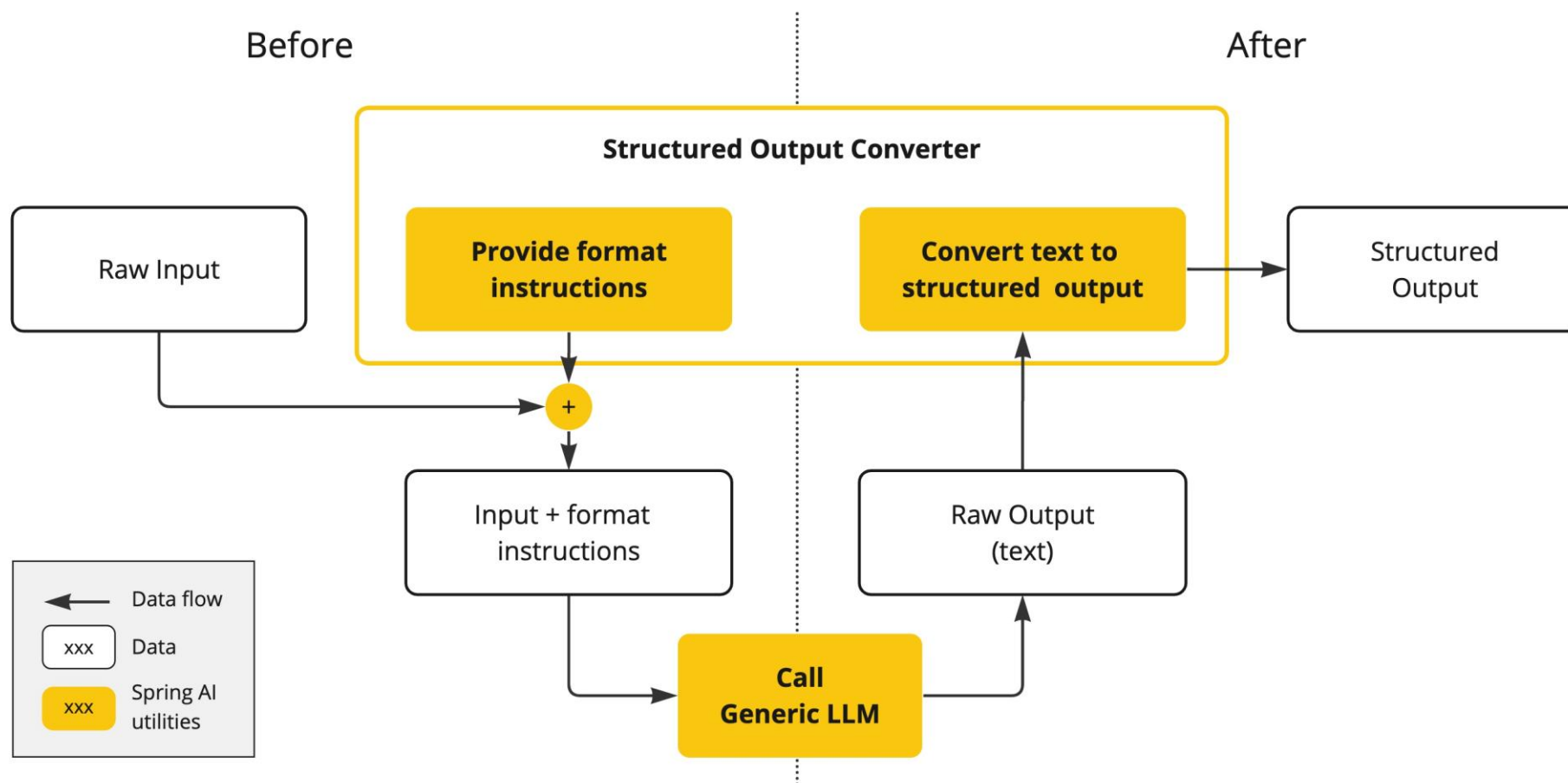
Structured Output

What is it?

- Plain text responses often not useful
- Structured Output generates Java objects

Structured Output

Principle



Structured Output

Code Example

```
@RestController
public class RecipeController {
    private final ChatClient chatClient;

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    record Recipe(String name,
                  String description,
                  List<String> ingredients,
                  String instructions) {}

    @GetMapping("recipe/json")
    public Recipe recipeJson() {
        return chatClient.prompt()
            .user("Give me a recipe for a delicious dish")
            .call()
            .entity(Recipe.class);
    }
}
```

Images

Images

Image to Text

```
@RestController
public class RecipeController {
    private final ChatClient chatClient;

    @Value("classpath:/test.jpg")
    private Resource image;

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    @GetMapping("/recipe/image/describe")
    public String recipeImageDescribe() {
        return chatClient
            .prompt()
            .user(
                PromptUserSpec u ->
                    u.text("What dish is this?")
                    .media(MimeTypeUtils.IMAGE_JPEG, image))
            .call()
            .content();
    }
}
```

Images

Text to Image

```
@RestController
public class RecipeController {
    private final ImageModel imageModel;

    public RecipeController(ImageModel imageModel) {
        this.imageModel = imageModel;
    }

    @GetMapping("recipe/image/create")
    public Image recipeImage() {
        ImageResponse response =
            imageModel.call(
                new ImagePrompt(
                    "Generate an image of a delicious pasta dish",
                    ImageOptionsBuilder.builder().N(1).height(1024).width(1024).build()));

        return response.getResult().getOutput();
    }
}
```

Audio

Audio

Audio to Text

```
@RestController
@RequestMapping("/audio-text")
public class AudioTextController {
    private final ChatModel chatModel; 2 usages

    public AudioTextController(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping
    public String getAudioText() {
        var audioResource = new ClassPathResource("sample.mp3");

        var userMessage = UserMessage.builder().text(textContent: "What is this recording about?")
            .media(List.of(new Media(MimeTypeUtils.parseMimeType("audio/mp3"), audioResource)))
            .build();

        ChatResponse response = chatModel.call(new Prompt(List.of(userMessage),
            OpenAiChatOptions.builder().model(OpenAiApi.ChatModel.GPT_4_0_AUDIO_PREVIEW).build()));

        return response.getResult().getOutput().getText();
    }
}
```

Audio

Text to Audio

```
@RestController
@RequestMapping("/text-audio")
public class TextAudioController {
    private final ChatModel chatModel; 2 usages

    public TextAudioController(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping
    public ResponseEntity<byte[]> getAudio(@RequestParam("q") String query) {
        var userMessage = new UserMessage(query);

        ChatResponse response = chatModel.call(new Prompt(
            List.of(userMessage),
            OpenAiChatOptions.builder()
                .model(OpenAiApi.ChatModel.GPT_4_0_AUDIO_PREVIEW)
                .outputModalities(List.of("text", "audio"))
                .outputAudio(new AudioParameters(AudioParameters.Voice.ALLOY, AudioParameters.AudioResponseFormat.WAV))
                .build()
        ));

        byte[] waveAudio = response.getResult().getOutput().getMedia().get(0).getDataAsByteArray();

        String fileName = "response.wav";

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.parseMediaType("audio/wav"));
        headers.setContentDisposition(ContentDisposition.builder("type: attachment").filename(fileName).build());

        return ResponseEntity.ok()
            .headers(headers)
            .body(waveAudio);
    }
}
```

Data Enrichment

A conceptual image for data enrichment. It features a smartphone in the center, tilted diagonally. The screen shows a data visualization with a network graph overlay, consisting of interconnected nodes and lines. The background is dark with a subtle pattern of glowing blue nodes and lines, suggesting a digital or network environment.

Data Enrichment

Adding additional information

- How can you equip the AI model with information on which it has not been trained?
- Tailoring and fine tuning the model -> Very Challenging
- Tool Calling
- Prompt Stuffing

Data Enrichment

Tool Calling

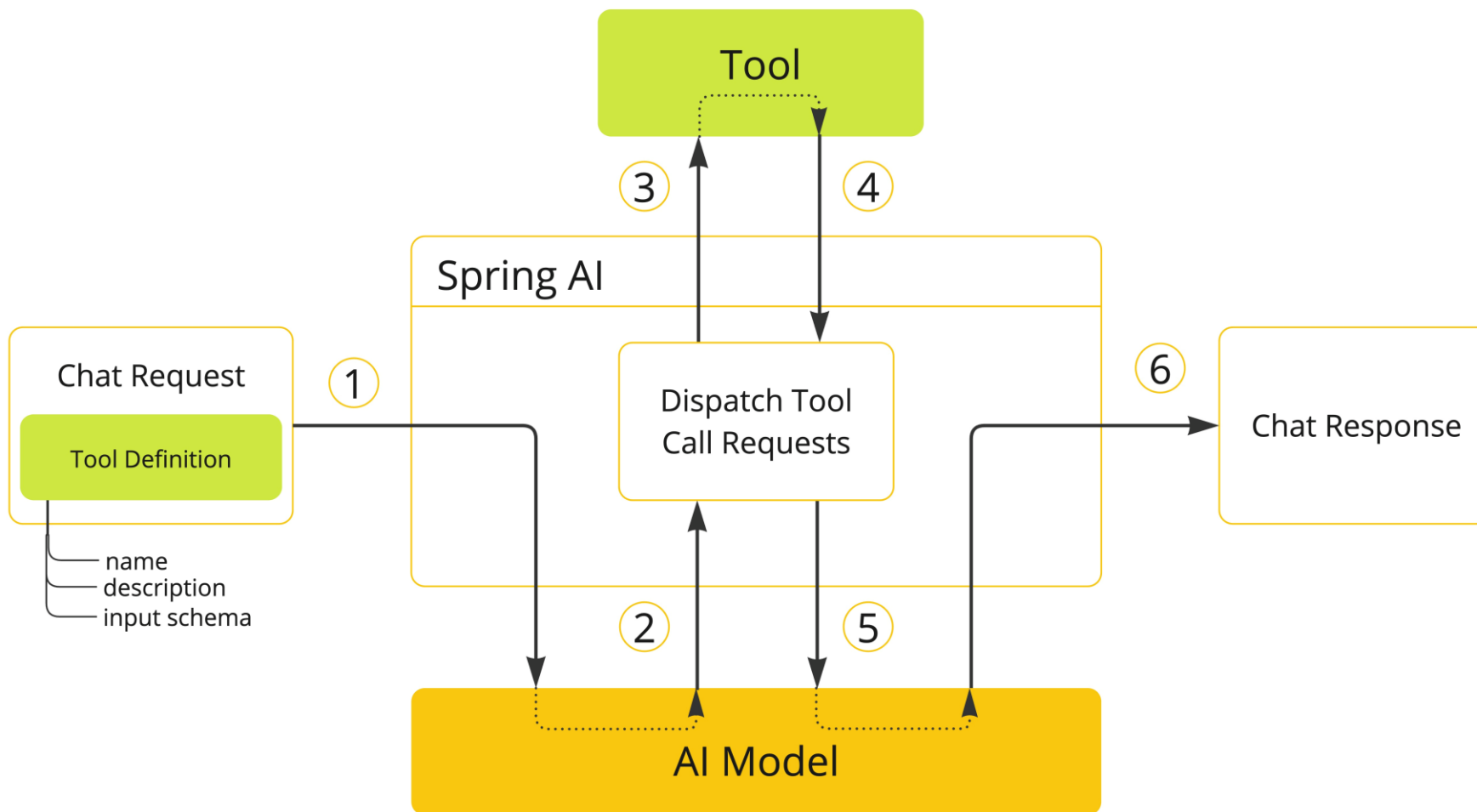
Data Enrichment

Tool Calling

- Provides the model with callable functions
- Model decides which functions to call

Data Enrichment

Tool Calling



Data Enrichment

Tool Calling

```
@RestController
public class RecipeController {
    private final ChatClient chatClient;

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder.build();
    }

    @GetMapping("/recipe/weather")
    public String recipeWeather() {
        return chatClient.prompt(
            "Give me a recipe for the current weather "
        ).tools(new WeatherTools())
        .call()
        .content();
    }
}

public class WeatherTools {
    @Tool(name = "weather", description = "Get the current weather")
    String getWeather(String location) {
        // Simulate a weather API call
        return "The weather in " + location + " is sunny with a high of 25°C.";
    }

    @Tool(name = "location", description = "Get the user's location")
    String getLocation() {
        // Simulate fetching the user's location
        return "Südtirol";
    }
}
```

Data Enrichment

Tool Calling

```
@RestController
public class RecipeController {
    private final ChatClient chatClient; 2 usages

    public RecipeController(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder
            .defaultTools(new PreferenceTool())
            .build();
    }

    @GetMapping("/recipe")
    public String recipe() {
        return chatClient
            .prompt() ChatClientRequestSpec
            .user( text: "Give me a recipe for a delicious dish")
            .call() CallResponseSpec
            .content();
    }

    public class PreferenceTool { 1 usage
        @Tool(name = "preference", description = "Get user preference for recipes")
        String getPreference() {
            // Simulate fetching user preferences from a database or configuration
            return "German cuisine";
        }
    }
}
```

Data Enrichment

Prompt Stuffing

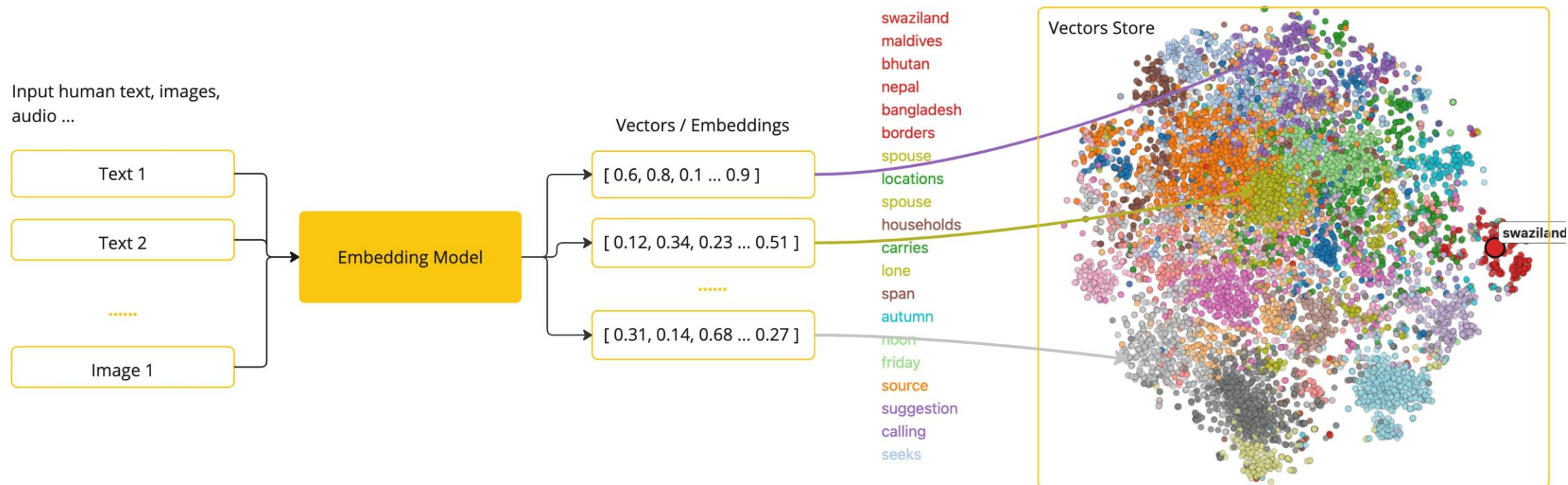
Data Enrichment

Prompt Stuffing

- Adding information to the prompt
- Information can be stored into a vector database

Data Enrichment

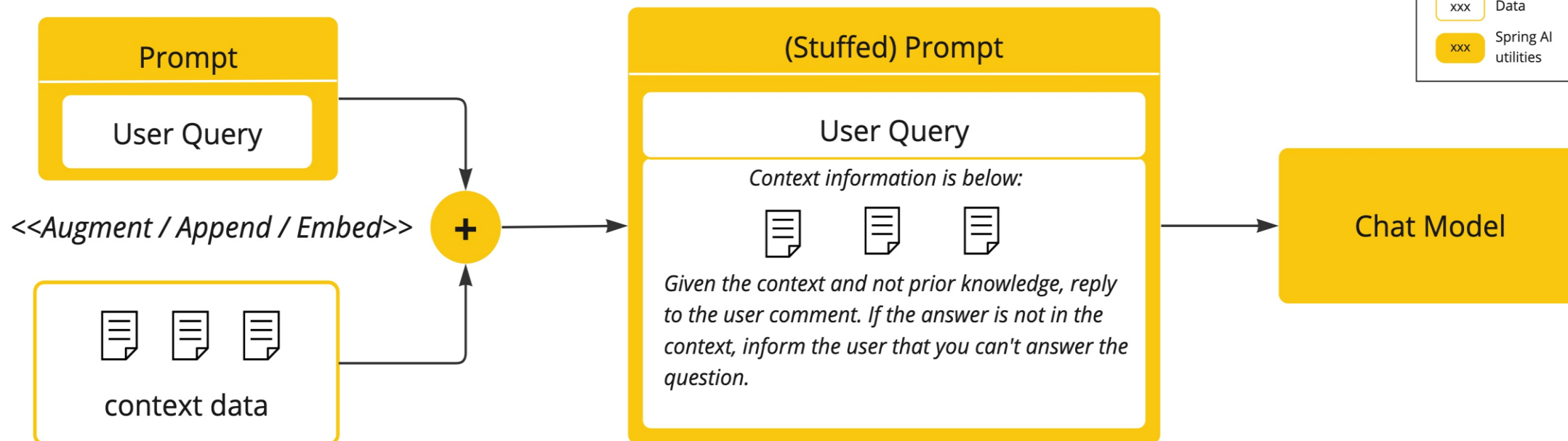
Vector Databases



Data Enrichment

Prompt Stuffing

Prompt Stuffing



Data Enrichment

Vector Database

```
@RestController
@RequestMapping("/vector-init")
public class VectorInitController {
    private final VectorStore vectorStore; 2 usages

    public VectorInitController(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    @GetMapping
    public String initVector() {
        List<Document> documents = List.of(
            new Document(text: "Spring AI rocks!! Spring AI rocks!! Spring AI rocks!! Spring AI rocks!! Spring AI rocks!!", Map.of(k1: "meta1", v1: "meta1")),
            new Document(text: "Pasta Carbonara is a traditional Italian dish made with eggs, cheese, pancetta, and pepper.", Map.of(k1: "meta1", v1: "meta1")),
            new Document(text: "You walk forward facing the past and you turn back toward the future.", Map.of(k1: "meta2", v1: "meta2")));

        vectorStore.add(documents);

        return "initialized " + documents.size();
    }
}
```

Data Enrichment

Retrieving documents

```
@RestController
@RequestMapping("/vector-topk")
public class VectorTopKController {
    private final VectorStore vectorStore; 2 usages
    private final ChatClient chatClient; 2 usages

    public VectorTopKController(VectorStore vectorStore, ChatClient.Builder builder) {
        this.vectorStore = vectorStore;
        this.chatClient = builder.build();
    }

    @GetMapping
    public String getTopK(@RequestParam(name = "q", defaultValue = "recipe for carbonara pasta") String userQuery) {
        VectorStoreDocumentRetriever retriever = VectorStoreDocumentRetriever.builder()
            .vectorStore(vectorStore)
            .topK(1)
            .build();

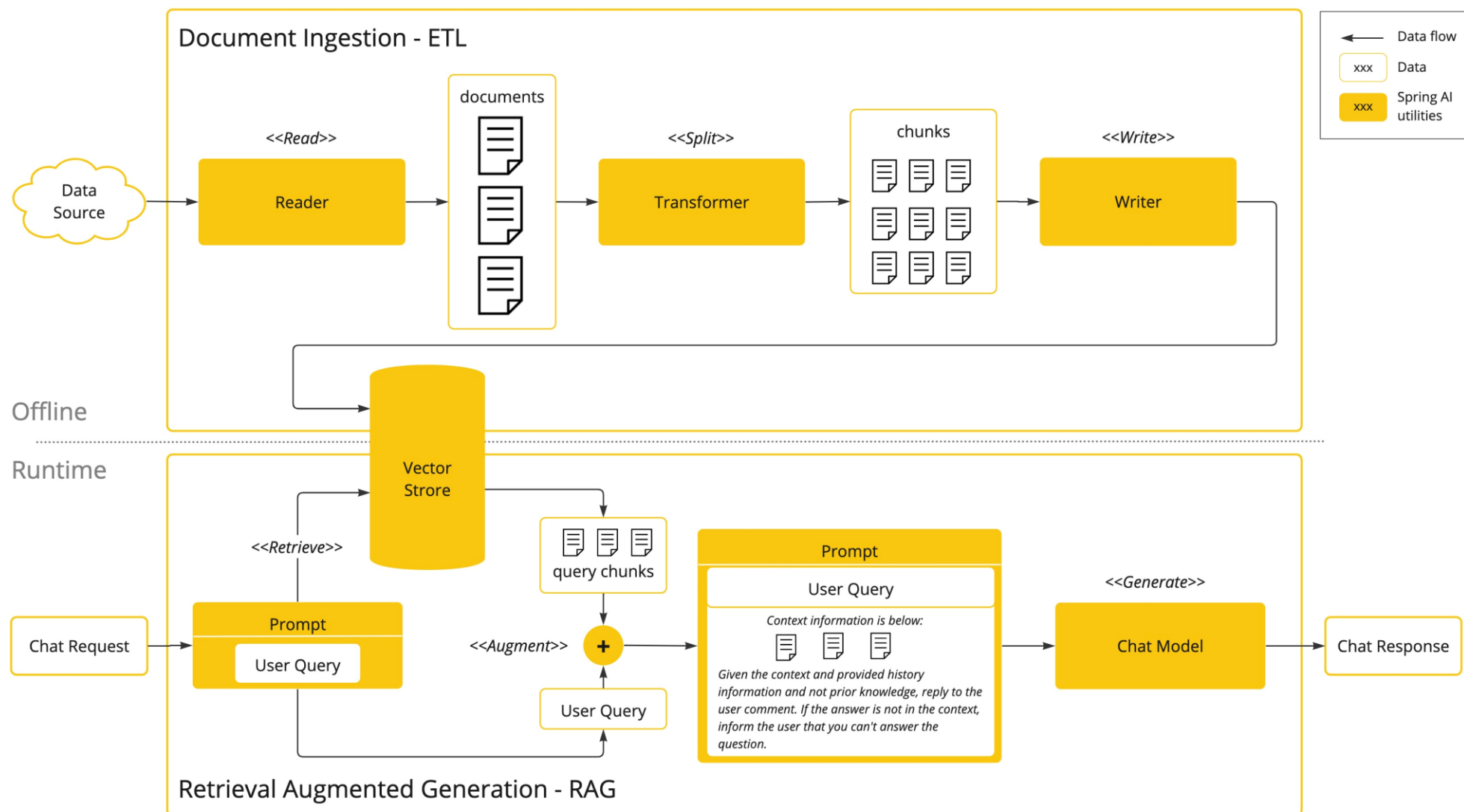
        List<Document> retrievedDocs = retriever.retrieve(new Query(userQuery));

        String formattedDocuments = retrievedDocs.stream() Stream<Document>
            .map(Document::getText) Stream<String>
            .reduce(identity: "", (String a, String b) -> a + "\n--\n" + b);

        return chatClient.prompt() ChatClientRequestSpec
            .user(text: "Using this document as additional information, provide a detailed answer to the query: " + userQuery + "\n\nDocument:\n" + formattedDocuments)
            .call() CallResponseSpec
            .content();
    }
}
```

Data Enrichment

Retrieval Augmented Generation



Data Enrichment

Document Ingestion

```
@RestController
@RequestMapping("/vector-pdf")
public class VectorPdfController {
    private final VectorStore vectorStore; 2 usages

    public VectorPdfController(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    @GetMapping
    public String processPDF() {
        var reader = new PagePdfDocumentReader(resourceUrl: "sample.pdf");

        var splitter = new TokenTextSplitter();

        List<Document> documents = splitter.apply(reader.get());

        vectorStore.add(documents);

        return "processed " + documents.size() + " documents";
    }
}
```

Data Enrichment

Retrieval Augmented Generation

```
@RestController
@RequestMapping("/vector-query")
public class VectorQueryController {
    private final ChatClient chatClient; 2 usages

    public VectorQueryController(VectorStore vectorStore, ChatClient.Builder chatClientBuilder) {
        Advisor retrievalAugmentationAdvisor = RetrievalAugmentationAdvisor.builder()
            .documentRetriever(VectorStoreDocumentRetriever.builder()
                .similarityThreshold(0.50)
                .vectorStore(vectorStore)
                .build())
            .build();

        this.chatClient = chatClientBuilder
            .defaultAdvisors(retrievalAugmentationAdvisor)
            .build();
    }

    @GetMapping
    public String ragQuery(@RequestParam(name = "q", defaultValue = "what is carbonara") String userQuery) {
        return chatClient.prompt() ChatClientRequestSpec
            .user(userQuery)
            .call() CallResponseSpec
            .content();
    }
}
```

Observability

Observability

What is it?

- Means to check what happens behind the scenes
- Metrics to measure model usage

Observability

Application Properties

```
spring.ai.chat.client.observations.log-prompt=true  
spring.ai.chat.observations.log-prompt=true  
spring.ai.chat.observations.log-completion= true  
spring.ai.image.observations.log-prompt= true  
spring.ai.chat.observations.include-error-logging=true
```

Observability

Actuators

```
"names": [  
    "application.ready.time",  
    "application.started.time",  
    "disk.free",  
    "disk.total",  
    "executor.active",  
    "executor.completed",  
    "executor.pool.core",  
    "executor.pool.max",  
    "executor.pool.size",  
    "executor.queue.remaining",  
    "executor.queued",  
    "gen_ai.client.operation",  
    "gen_ai.client.operation.active",  
    "gen_ai.client.token.usage",  
    "http.client.requests",  
    "http.client.requests.active",  
    "http.server.requests.active",  
    "jvm.buffer.count",  
    "jvm.buffer.memory.used",  
    "jvm.buffer.total.capacity",  
    "jvm.classes.loaded",  
    "jvm.classes.unloaded",  
    "jvm.compilation.time",  
    "jvm.gc.live.data.size",  
    "jvm.gc.max.data.size",  
    "jvm.gc.memory.allocated",  
    "jvm.gc.memory.promoted",  
    "jvm.gc.overhead",  
    "jvm.gc.pause",  
    "jvm.info",  
    "jvm.memory.committed",  
    "jvm.memory.max",  
    "jvm.memory.usage.after.gc",  
    "jvm.memory.used",  
    "jvm.threads.daemon",  
    "jvm.threads.live",  
    "jvm.threads.peak",  
    "jvm.threads.started",  
    "jvm.threads.states",  
    "logback.events",  
    "process.cpu.time",  
    "process.cpu.usage",  
    "process.start.time",  
    "process.uptime",  
    "spring.ai.advisor.active",  
    "spring.ai.chat.client.active",  
    "spring.ai.tool",  
    "spring.ai.tool.active",  
    "system.cpu.count",  
    "system.cpu.usage",  
    "tomcat.sessions.active.current",  
    "tomcat.sessions.active.max",  
    "tomcat.sessions.alive.max",  
    "tomcat.sessions.created",  
    "tomcat.sessions.expired",  
    "tomcat.sessions.rejected"  
]
```

Observability

Console Output

```
2025-11-04T15:59:08.831+01:00 INFO 26920 --- [mampf] [nio-8080-exec-1] hatClientPromptContentObservationHandler : Chat Client Prompt Content:
["user": "  Evaluate whether or not the following claim is supported by the provided document.
  Respond with "yes" if the claim is supported, or "no" if it is not.
  Document: \n Carbonara is a traditional Italian pasta dish made with eggs, cheese (Pecorino Romano), guanciale (cured pork cheek), and black pepper.
The main ingredients of pasta carbonara are pasta, eggs, Pecorino Romano cheese, guanciale, and black pepper. Cream is NOT traditionally used in authentic carbonara.
Guanciale is the traditional meat used in carbonara, though pancetta is sometimes used as a substitute.\n
  Claim: \n The main ingredient in traditional carbonara is guanciale, which is cured pork cheek or jowl. Other key ingredients typically include pasta (usually spaghetti), eggs, Pecorino Romano cheese, and black pepper.
"]
```

Evaluation Testing

Evaluation Testing

What is it?

- Traditional Tests often not enough
- Provides means to test generated results

Evaluation Testing

Relevancy Evaluator

```
public RecipeController(ChatClient.Builder chatClientBuilder) {
    this.chatClientBuilder = chatClientBuilder;
    this.chatClient = chatClientBuilder.build();
}

@GetMapping(@PathVariable("recipe-with-evaluation"))
public RecipeEvaluation recipeWithEvaluation(@RequestParam(defaultValue = "a delicious dish") String dish) {
    String userQuery = "Give me a recipe for " + dish;

    String aiResponse = chatClient
        .prompt() ChatClientRequestSpec
        .user(userQuery)
        .call() CallResponseSpec
        .content();

    RelevancyEvaluator evaluator = new RelevancyEvaluator(chatClientBuilder);

    EvaluationRequest evaluationRequest = new EvaluationRequest(
        userQuery,
        List.of(),
        aiResponse
    );

    EvaluationResponse evaluationResponse = evaluator.evaluate(evaluationRequest);

    return new RecipeEvaluation(
        dish,
        aiResponse,
        evaluationResponse.getScore(),
        evaluationResponse.isPass()
    );
}

public record RecipeEvaluation( 2 usages
    String dishRequested, no usages
    String recipe, no usages
    double relevancyScore, no usages
    boolean isRelevant no usages
) {}
```

Evaluation Testing

Fact Checking Evaluator

```
@GetMapping(Ⓢ"recipe-fact-check")
public FactCheckEvaluation recipeFactCheck(
    @RequestParam(defaultValue = "What is the main ingredient in carbonara?") String question) {

    String aiResponse = chatClient
        .prompt() ChatClientRequestSpec
        .user(question)
        .call() CallResponseSpec
        .content();

    List<Document> supportingFacts = List.of(
        new Document(content: "Carbonara is a traditional Italian pasta dish made with eggs, cheese (Pecorino Romano), guanciale (cured pork cheek), and black pepper."),
        new Document(content: "The main ingredients of pasta carbonara are pasta, eggs, Pecorino Romano cheese, guanciale, and black pepper. Cream is NOT traditionally used in authentic carbonara."),
        new Document(content: "Guanciale is the traditional meat used in carbonara, though pancetta is sometimes used as a substitute.")
    );

    FactCheckingEvaluator evaluator = new FactCheckingEvaluator(chatClientBuilder);

    EvaluationRequest evaluationRequest = new EvaluationRequest(
        question,
        supportingFacts, // The factual information to check against
        aiResponse
    );

    EvaluationResponse evaluationResponse = evaluator.evaluate(evaluationRequest);

    return new FactCheckEvaluation(
        question,
        aiResponse,
        evaluationResponse.getScore(),
        evaluationResponse.isPass()
    );
}
```

Benefits & Caveats

Benefits & Caveats

Benefits

- Easy access to AI features in traditional Spring Applications
- Provider Agnostic
- Type safety for generated content
- New possibilities for better UX:
 - Voice controlled Applications with single point of entry
 - Content generation on demand
 - Dynamic UI arrangement
 - Moderation for user content
 - Ecc.

Benefits & Caveats

Caveats

- Higher maintenance than out-of-the-box solutions
- Some model / provider dependency remains
- Testing is still a work in progress

” Just because we can build it
doesn't mean we should

AI usage has real environmental costs

Resources

Examples & Documentation

- <https://github.com/Jjemus/spring-ai-examples>
- <https://spring.io/projects/spring-ai>